

Class 1 - Web Fundamentals and Intro to HTML

Agenda

1. Overview of the Curriculum
2. How the Web Works
3. Setting up VS Code
4. Introduction to HTML (the skeleton of the web).
5. HTML Tags (headings, paragraphs, images, links, buttons).
6. Semantic Tags (making web pages meaningful).

Curriculum Overview

Class-wise Breakup with Projects

1. **HTML/CSS** (7 classes) → Portfolio Website
2. **JavaScript + Machine Coding (DOM)** (22 classes) → Kanban Board + Mini Projects
3. **React** (13 classes) → Movies App
4. **MERN** (12–14 classes) → BookMyShow Clone

Reflection:

- *every framework (React, Angular, Vue) is just a smarter way of writing HTML, CSS, and JavaScript. If you master the basics, you can learn any framework quickly.*

HTML / CSS

- **HTML is the skeleton** of every website. It gives structure — headings, paragraphs, images, links.
- **CSS is the paint and interior design.** It makes the structure look good, readable, and user-friendly.

- Together, they form the *blueprint* of the web.

JavaScript

- With JS, your site can react: button clicks, animations, hiding/showing sections, validations, etc.

Reflection:

Every time you click 'Add to Cart' on Amazon, a small piece of JavaScript is firing. No JS, no shopping cart

How Web Works

What happens when you type `www.scaler.com` in your browser and press Enter?

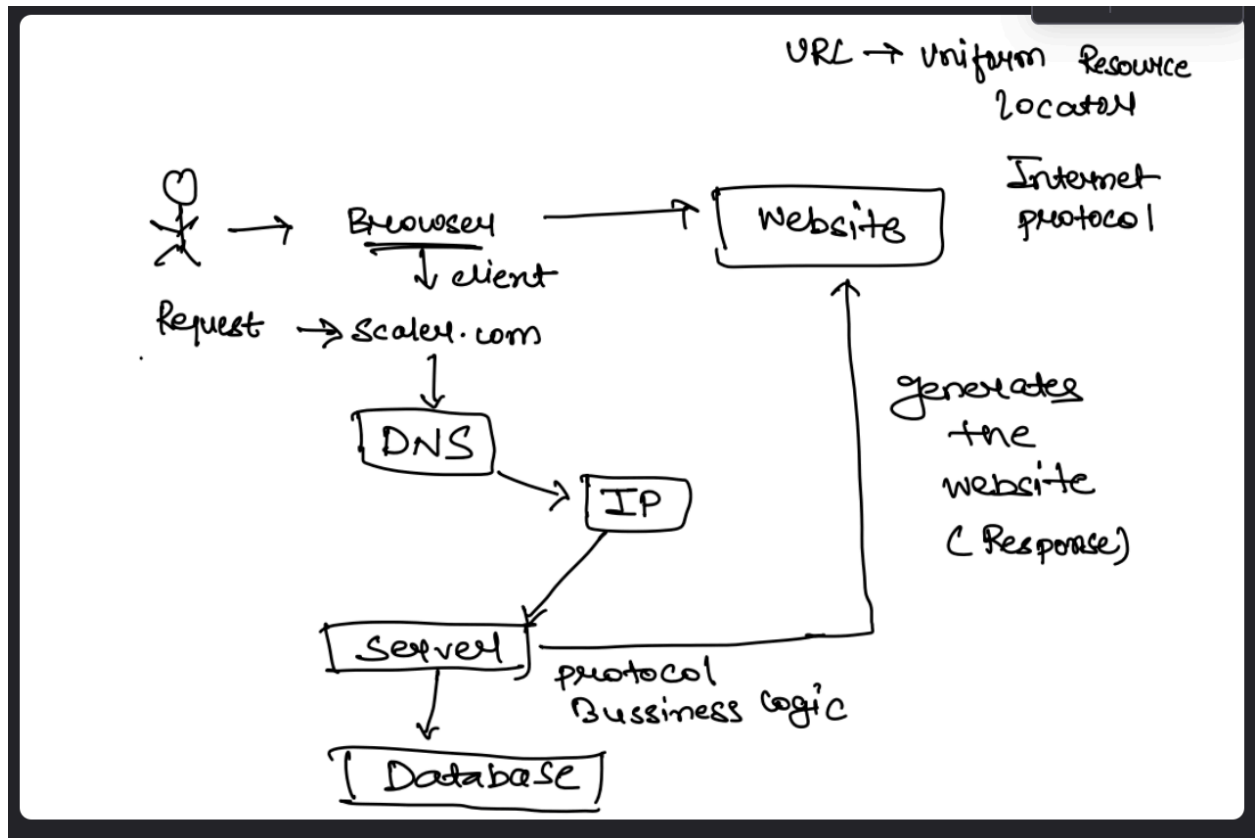
different **actors**:

- **Client** → That's you (your browser, your laptop, your phone).
- **Server** → The computer somewhere in the world that holds the website.
- **Internet connection** → The roads and highways connecting clients to servers.
- **DNS (Domain Name System)** → The phonebook that helps your computer find the right server.
- **HTTP** → The common language spoken between client and server.

Reflection #1: Internet as a Global Nervous System

- The internet is powered by **fibre optic cables under the sea** that carry data almost at the speed of light.
- submarinecablemap.com

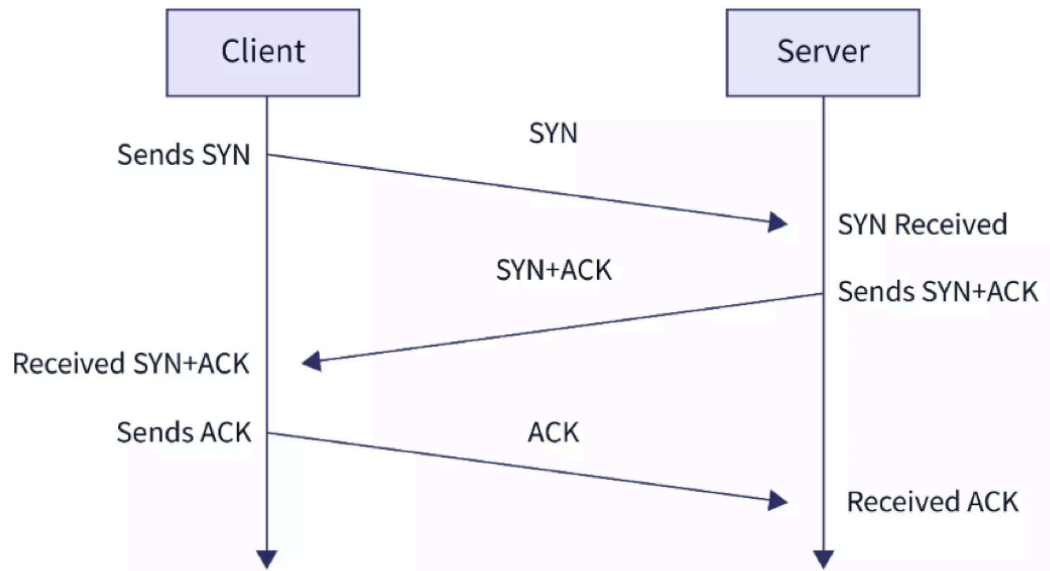
Process



The First Conversation (TCP/IP Handshake)

Before the actual data exchange happens - an agreement to exchange data is formed

This is called the **TCP 3-way handshake**.



a.

DNS Resolution & Caching

When you type `scaler.com`, your computer first checks closer to home:

1. **Browser cache** → Chrome/Firefox may have already stored the IP from a past visit.
 - open `chrome://net-internals/#dns` (or in new versions, `chrome://dns`) to show cached domains.
2. **OS cache** → If browser doesn't have it, your operating system may have saved it.
3. **Router cache** → Your home router also remembers common sites to speed things up.
4. **ISP cache** → Your Internet Service Provider keeps a big record of popular websites.

Step 2: Going Global

If none of these caches have the IP, then the real detective work begins.

- The request goes to one of the **13 root servers** of the internet.
(13 logical root servers hold the master records for the entire internet.)
- The root server doesn't know the exact IP of `scaler . com`, but it says:
I know who manages . com domains – go ask the TLD server.
- The **TLD (Top Level Domain) server** for `. com` points you to the **authoritative name server** of `scaler . com`.
- Finally, the authoritative server says: *'Here's the exact IP address for Scaler.*

Reflection:

- Even though there are billions of websites, only **13 logical root servers** hold the keys to them all. If they ever went down, large parts of the internet would stop resolving.

Installation and Introduction to HTML

Setting Up the Tools (VS Code)

1. IDE (Integrated Development Environment):

- Explain:. We need a place where we can write, run, and test code easily.
- We'll use **Visual Studio Code (VS Code)** – it's free, lightweight, and developer-friendly.

2. Steps:

- Download from <https://code.visualstudio.com/>
- Open VS Code → Create a new folder (name it after your batch).
- Inside the folder, create a file: `index . html`.

First HTML Page (Boilerplate)

In `index . html`, type `!` and hit Enter → VS Code generates **boilerplate HTML**.

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>HTML Demo</title>
</head>
<body>
  <h1>Hello, World!</h1>
</body>
</html>
```

1. **<!DOCTYPE html>**

- Tells the browser this is an HTML5 page.
- Without this, the browser might fall back to quirks mode and interpret things incorrectly.

2. **<html lang="en">**

- The root container for the page.
- The `lang="en"` attribute tells search engines and screen readers that the content is in English.
- Accessibility tip: This helps assistive technologies (like screen readers for visually impaired users) pronounce words correctly.

3. **<head>**

- Think of this as the *brain* of the page.
- Contains metadata (data about the page) that isn't shown directly on screen but is crucial.

4. **Meta Tags:**

- `<meta charset="UTF-8">` → ensures support for multiple languages and special characters (e.g., accents in French, Hindi script, Chinese characters).
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">` → makes the page responsive for mobile devices.
- `<title>` → sets the text on the browser tab.

5. **Insight:** If you search a site on Google, the snippet and title you see often come from these meta tags.
6. **<body>**
 - The *body* of the page — what users actually see (text, images, buttons).
 - If <head> is the brain, <body> is the face and limbs.

Step 4: Running the Page

- Write Hello World inside <body>.
- Open the file by double-clicking or using **Live Server extension** for instant refresh.

Step 5: Case Insensitivity in HTML

- HTML tags are **case-insensitive**. <BODY>, <Body>, <body> — all work the same.
- **But best practice** is to use lowercase for readability and consistency.

HTML Tags

HTML is all about **tags**.

Think of tags as *labels inside angled brackets* (< >) that tell the browser how to treat content.

1. Headings (<h1> to <h6>)

- <h1> is the most important (page title), <h6> the least.
- Browsers, search engines, and screen readers use this hierarchy.

```
<h1>Breaking Bad</h1>
<h2>Amazing prison escape series</h2>
<h6>Thrilling series to keep ou on your toes</h6>
<p>exciting series to watch for</p>
```

2. Paragraph (<p>)

- For normal text blocks.

- Use Emmet shortcut: type `lorem` + Tab to generate dummy text.
- **Insight:** Search engines scan `<p>` content heavily when ranking a page.

3. Images (`<img>`)

- Self-closing tag (no `</img>`).
- Attributes:
 - `src` → source of image.
 - `alt` → alternative text (important for accessibility and SEO).

```

```

Analogy: Attributes are like adjectives describing a noun.

4. Links (`<a>`)

- Anchor tag used to create hyperlinks.
- Attributes:
 - `href` → the destination.
 - `target="_blank"` → opens in new tab.

```
<a href="https://www.google.com">Click here to go to google</a>
```

```
<a href="https://www.google.com" target="_blank"  
>Click here to go to google</a  
>
```

5. Buttons (`<button>`)

- Represents a clickable action.
- By default, it doesn't "do" much until JS is added — but it looks like a button.

```
<button type="button">Download Resume</button>
```

6. Comments

- For leaving notes inside code (ignored by browser).

```
<!-- This is a comment -->
```

Structure Code with <div>

We need a way to **organize**. That's where <div> comes in.

1. What is <div>?

- <div> = "division" → an invisible box to group content.
- Doesn't add meaning (like headings or paragraphs) → but helps structure the layout.

```
<body>

  <div>

    <h1>Welcome to Scaler</h1>

    <p>We're glad you're here!</p>

  </div>

  <div>

    <a href="https://www.scaler.com" target="_blank">Go to Scaler</a>

  </div>
```

- Show how wrapping content in <div> lets us move whole sections together later with CSS.
- Nest <div>s to show hierarchy. - inside rooms you might have study area, resting area, gaming setup

Github

We'll use **GitHub** — the world's most popular platform for hosting and collaborating on code.

1. What is GitHub?

- Think of it as a **global online library of code**.
- Every project is like a book in the library.
- GitHub keeps track of every edit — who made it, when, and why.
- Developers across the world use it to collaborate, review, and showcase their work.

Imagine Google Docs but for code — multiple people can work on the same project, track changes, and revert if something breaks.

- I will maintain a **class repository** → all the code we write together will be pushed there.
- You can always refer to it if you miss something.
- This will also help you build the habit of version control — a must for real-world projects.

3. What You Need to Do

1. Sign up on <https://github.com>.
 - Choose a professional username (preferably your real name).
2. Verify your email.
3. Once logged in, you're done for today.

Scaler recording for GIT - <https://www.scaler.com/meetings/i/basics-of-git-and-github/archive>

Semantic Tags.

1. What Are Semantic Tags?

- Tags that describe the **role** of content.
- Introduced in **HTML5** to improve readability, accessibility, and SEO.

Examples:

- `<header>` → top section, usually logo + nav + intro.
- `<nav>` → navigation bar.
- `<section>` → logical sections of the page.
- `<article>` → self-contained content (e.g., blog post).
- `<footer>` → bottom section with links, copyright.

2. Why Do They Matter?

1. Readability for Developers:

- Easier to understand when revisiting code.
- `<header>` tells you exactly what that block is.

2. Accessibility for Users:

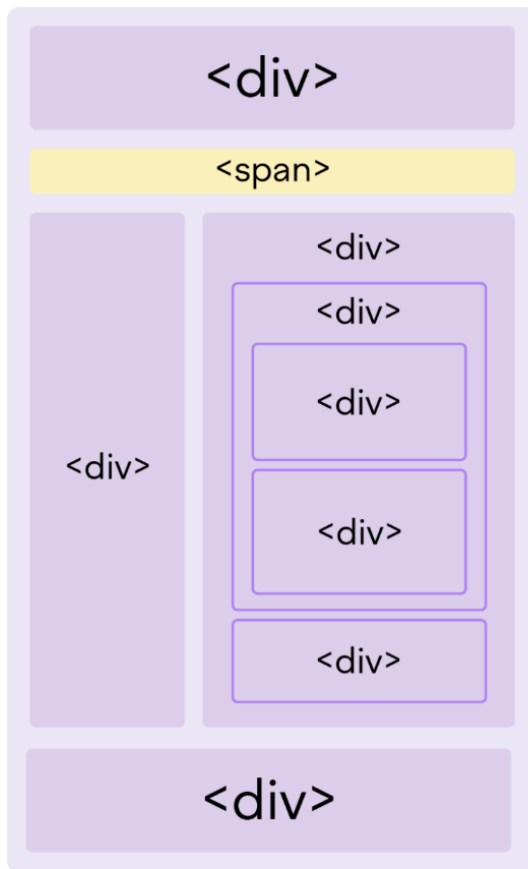
- Screen readers can jump between `<header>`, `<nav>`, `<section>`.
- Helps visually impaired users navigate faster.

3. SEO (Search Engines):

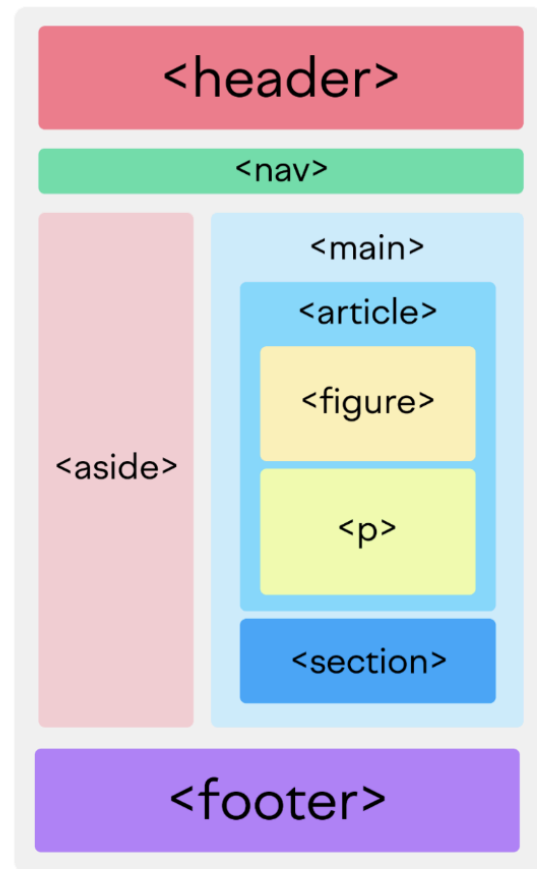
- Search engines give more importance to content inside semantic tags.

Insight: Google's crawler actually *skips over random `<div>`s*, but pays special attention to `<header>`, `<article>`, etc.

Non-Semantic HTML



Semantic HTML



https://www.w3schools.com/html/html5_semantic_elements.asp

```
<body>

  <header>

    <nav>

      <a href="#">Home</a>

      <a href="#">About</a>

      <a href="#">Projects</a>
```

```
<a href="#">Contacts</a>

</nav>

<div>

  <h1>John Doe</h1>

  <h2>Full stack developer</h2>

</div>

</header>

<main>

  <section>

    <h2>About me</h2>

    <p>I am a beginner full stack developer working on AI MERN projects</p>

  </section>

  <section>

    <h2>

      Projects

    <ul>

      <li>AI Chatbot</li>

      <li>AI Image generator</li>

      <li>AI Code generator</li>

    </ul>

    </h2>

  </section>

</main>
```

```
<footer>Copyright ©2025 John Doe</footer>  
  
</body>
```

Copyright - option + g ©

4. Preparing for the Portfolio

- Replace <div>-based layout with **semantic structure**.